Reuse an existing abstraction if the repository already has one.

…ceptually:

````
```text
Bud / Orbit / Agents
        ↓
AI Service / Capability Layer
        ↓
Provider Adapter
        ↓
External AI Provider
```
````

The service should support at least:

````
```text
generateText()
generateStructured()
generateImage()
```
````

Provider/model configuration must use server-side environment variables. Do not hard-code secrets.

Use an architecture equivalent to:

````
```text
AI_PROVIDER=
AI_API_KEY=
AI_TEXT_MODEL=
AI_REASONING_MODEL=
AI_IMAGE_MODEL=
```
````

Use the repository's existing provider where appropriate, but keep the adapter replaceable so the application is not architecturally tied to one vendor.

6. NETLIFY ARCHITECTURE

Privileged AI calls must happen server-side.

vendor.

9:40 LTE 19

NETLIFY ARCHITECTURE

Privileged AI calls must happen server-side.

Share Message
Done. I made two versions so you can either down...

```
Student Browser
      ↓
Netlify frontend
      ↓
Netlify Function / secure server endpoint
      ↓
AI Service
      ↓
Provider Adapter
      ↓
External AI API
```

Never ship AI API keys to the browser.

Create or repair Netlify functions/endpoints as required by the actual framework. Preserve the existing application architecture where possible rather than rewriting the whole project.

7. BUD CONVERSATION

Conversation must be a first-class intent.

Inputs such as:

- “Hello”
- “How are you?”
- “I have a girlfriend.”
- “I’m frustrated.”
- “Why?”
- “Tell me more.”
- “Okay.”
- “Yes.”

- “Why?”
- “Tell me more.”
- “Okay.”
- “Yes.”

must be capable of being handled directly by Bud when no specialist is required.

Do not force ordinary conversation through Scholar.

Preserve conversation history/context so short follow-ups resolve against the active conversation/workspace.

Bud must answer naturally, clearly and directly. Do not expose internal agent names or routing details.

8. BUD IMAGE GENERATION

Image generation must be real, not simulated.

For example:

“Generate an image of a university campus at sunset.”

must be able to follow:

````
```text
Student
 ↓
Bud
 ↓
Guardian
 ↓
Orbit/capability routing
 ↓
AIProvider.generateImage()
 ↓
real image result
 ↓
````
````

9:40 LTE 19

```
Orbit/capability routing
↓
AIProvider.generateImage()
↓
l image result
↓
Bud
```

Share Message
Done. I made two versions so you can either down...

Never fabricate an image URL. Never claim success if the provider failed.

If the current Message model only supports text, extend it safely to support media, preserving old messages. Prefer a media structure such as:

```
```text
media: [
  { type, url, mime_type, alt }
]
```
```

or an equivalent existing project convention.

Generated/uploaded media must not automatically become student memory.

9. ORBIT

Orbit remains the sole orchestrator.

Implement intent-aware routing so the system does not wake every specialist for every request.

Examples:

```
```text
Conversation:
Bud → direct response

Academic:
Bud → Guardian → Orbit → Scholar → Orbit
```
```

```
Conversation:
Bud → direct response

Academic:
Bud → Guardian → Orbit → Scholar → Orbit
→ Bud

Planning:
Bud → Guardian → Orbit → Coach → Orbit
→ Bud

Resources/discovery:
Bud → Guardian → Orbit → Atlas → Orbit
→ Bud

Multi-agent:
Bud → Guardian → Orbit → parallel/
sequential agents → Orbit → Bud

Image:
Bud → Guardian → Orbit/capability → image
provider → Bud
```

Agents must never directly invoke agents.

10. AGENT PERSONAL INDEPENDENCE

Every agent must have a real registry/runtime definition containing at least:

```text
id
name
description
role
system_instructions
capabilities
tools
permissions
context_requirements
memory_access
model_configuration
status
```

Each agent must be executable through a common runtime contract and must clearly define:

runtime contract and must clearly define:

- what it does;
- what it does not do;
- allowed context;

- expected input;
- expected output;
- failure behavior.

Standardize results, for example:

```
```text
{
  agent_id,
  status,
  result,
  evidence,
  confidence_internal,
  requested_actions,
  errors,
  metadata
}
```
```

Confidence remains internal. Never expose internal routing/agent mechanics to students.

Hard rule:

```
```text
Agent → recommends/results
Orbit → decides
Never: Agent → Agent
```
```

11. SPARK

Spark is a computation/execution substrate.

It may perform transformations, calculations or

perform transformations, calculations
tool execution required by the system, but it must
ecome a peer student-facing AI, an
estrator, or an agent-chain creator.

Do not redesign Spark into another specialist.

12. MEMORY, CONTEXT AND PRIVACY

Preserve the approved boundaries:

- Student Memory;
- Academic Notes;
- Project Context;
- Conversation History;
- Uploaded Documents;
- Saved Resources.

Do not blindly save conversations to memory.

Do not store transient current-world facts as student memory.

Keep student data strictly isolated.

Private academic/learning intelligence must never silently surface in public profiles, Communities, Connect/discovery, Messages, recommendations, or other social-facing content. This includes weaknesses, mistakes, learning patterns, progress signals and private Bud conversations.

13. CAPABILITY LAYER

Keep capabilities separate from the agent registry.

Conceptually:

```
```text
Agent Registry = who can reason/act
Capability Registry = what the system
```
```

```
Agent Registry = who can reason/act
Capability Registry = what the system
can do
```

Support capability definitions for text generation,

Do not activate frozen future agents prematurely merely because an interface exists.

14. RESEARCH / ORACLE

Oracle remains part of the 16-agent architecture but must remain frozen if the actual implementation phase still marks it frozen.

Do not pretend Oracle is production-active without implementation evidence.

When eventually activated, the intended deep-research architecture may use:

```text
Bud → Guardian → Orbit → Atlas → Scholar →
Oracle → Creator/Voice where needed →
Orbit → Bud
```

Do not fabricate sources, citations or verification.

15. REAL-TIME INFORMATION

If current-information capability exists, preserve and isolate it as a capability. If it does not, create a clean provider boundary without Base44.

Never pretend static model knowledge is live.

Do not turn current-information capability into a generic engagement/news feed.

- very clear language by default;
- university-level intelligence;
- simple without being childish;
- answer first when the answer is clear;
- ask only when clarification is genuinely necessary;
- adapt depth to the student's understanding;
- maintain conversational momentum;
- never shame the student;
- never expose internal agent names.

Confusion recovery:

```text
direct explanation
→ analogy
→ real-world example
→ smaller parts
→ identify what is unclear
```

Preserve the learning model:

```text
Exposure → Recall → Understanding →
Application → Mastery
```

When topic B is not understood, check whether a prerequisite A is missing rather than endlessly repeating B.

17. SAFETY AND EXECUTION LIMITS

Preserve Guardian as the safety/privacy/permission boundary.

Do not add Fixer or Therapist as agents.

Centralize configurable execution limits such as:

```text
MAX_EXECUTION_DEPTH
```

```
MAX_RETRIES_PER_AGENT
TOTAL_EXECUTION_TIME
```

Do not scatter arbitrary hard-coded limits throughout the code.

Do not make a generic "I hit a snag" response the default for every failure.

Distinguish technical failure types such as:

```text
provider_unavailable
provider_auth_error
provider_rate_limit
provider_quota
image_provider_error
invalid_response
routing_error
agent_error
synthesis_error
persistence_error
```

Log technical details server-side; return student-safe messages. Never expose secrets, stack traces or internal implementation details.

Do not falsely claim “unlimited AI.” The requirement is specifically removal of Base44 monthly integration dependence.

19. FIXER / VERIFICATION PRINCIPLE

Preserve the system-wide Fixer behavior:

```text
understand → break down → investigate →
connect → try → check → adjust → solve
```

Never claim success without evidence.